

ESP32 Smart RC Car using ESP-NOW (Automotive Lighting Automation)

PROJECT REPORT

Prepared By: Lovedeep Singh

Degree: B.Sc. Non-Medical (Science)

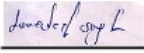
Domain: Embedded Systems | IoT | Automotive Electronics

Report Date: May 2026

Declaration

I, Lovedeep Singh, hereby declare that this project report titled "ESP32 Smart RC Car using ESP-NOW (Automotive Lighting Automation)" is an original work carried out by me for academic and learning purposes. All information and implementation in this report is based on my practical work and standard technical references.

Name: Lovedeep Singh

Signature: _____

Date: May 2026

Acknowledgement

I would like to express my gratitude to my teachers, mentors, and online technical communities for providing valuable guidance and learning resources. I also acknowledge the official ESP32 documentation and embedded systems references that supported the successful completion of this project.

Abstract

This project presents an ESP32-based Smart RC Vehicle designed to replicate realistic automotive control behavior using embedded systems and wireless communication. The vehicle is controlled using ESP-NOW, a low-latency peer-to-peer wireless protocol operating on the Wi-Fi hardware of ESP32. A transmitter ESP32 reads throttle and steering input from joystick modules and transmits real-time control packets to the receiver ESP32 installed on the vehicle.

The receiver ESP32 controls steering using a servo motor and speed using an Electronic Speed Controller (ESC). In addition to movement, the system includes realistic automotive lighting functions such as headlight control, hazard mode, brake light automation, and turn indicators with memory logic. Trim calibration is supported using EEPROM storage on the transmitter to correct joystick centering. The receiver includes steering center auto-calibration to improve stability. A failsafe mechanism has been implemented to stop the vehicle and activate hazard blinking during communication loss. The project demonstrates practical knowledge of embedded C++ programming, real-time control logic, wireless communication, and automotive-inspired automation.

Keywords: ESP32, ESP-NOW, Embedded Systems, RC Car, Servo Motor, ESC, Automotive Lighting

Table of Contents

1. Introduction
2. Objectives
3. System Overview
4. Hardware Requirements
5. Software Requirements
6. Communication Method (ESP-NOW)
7. Pin Configuration
8. Circuit Connections
9. Implementation Details
10. Control Logic and Automation Features
11. Testing and Results
12. Challenges Faced
13. Applications
14. Conclusion
15. Future Scope
16. References

1. Introduction

Embedded systems are widely used in modern automotive electronics for automation, safety, and control. Conventional RC vehicles mainly provide basic throttle and steering control. This project was developed to design an ESP32-based RC vehicle that demonstrates realistic automotive lighting automation and safety logic.

The ESP32 microcontroller was selected due to its powerful processor, PWM capability, and built-in wireless communication.

2. Objectives

- Develop low-latency wireless control using ESP-NOW.
- Implement servo steering and ESC throttle control.
- Implement headlights, hazard mode, indicators, and brake light automation.
- Implement indicator memory logic.
- Implement trim calibration using EEPROM.
- Implement failsafe mode during communication loss.
- Improve stability using steering center auto-calibration.

3. System Overview

The project consists of two ESP32 modules:

Transmitter Unit:

Reads joystick inputs (throttle and steering), reads headlight/hazard buttons, applies trim calibration, and transmits control packet using ESP-NOW.

Receiver Unit:

Receives packet and controls steering servo, ESC throttle, and automotive lighting functions in real-time.

4. Hardware Requirements

Transmitter:

- ESP32 Dev Board
- Two 1-axis joystick modules
- 6 buttons (4 trim + headlight + hazard)

Receiver:

- ESP32 Dev Board
- Servo motor (steering)
- ESC + DC motor
- LEDs for headlights, indicators, brake light
- Battery (Recommended: at least 2000mAh for smooth functioning)

Note: All modules must share a common ground.

5. Software Requirements

- Arduino IDE / PlatformIO
- Embedded C++
- Libraries: esp_now, WiFi, EEPROM, ESP32Servo

6. Communication Method (ESP-NOW)

ESP-NOW is a peer-to-peer wireless protocol by Espressif. It uses the ESP32 Wi-Fi radio but does not require router or internet. It provides low latency and reliable real-time control.

7. Pin Configuration

Receiver ESP32:

Steering Servo: GPIO25

ESC: GPIO26

Headlight: GPIO4

Front Left: GPIO13

Front Right: GPIO15

Rear Left: GPIO12

Rear Right: GPIO14

Brake Light: GPIO27

Transmitter ESP32:

Throttle Analog: GPIO32

Steering Analog: GPIO33

Trim Throttle Up: GPIO19

Trim Throttle Down: GPIO18

Trim Steering Up: GPIO23

Trim Steering Down: GPIO22

Headlight Button: GPIO4

Hazard Button: GPIO5

8. Implementation Details

Transmitter firmware reads analog values, applies border mapping, handles trim adjustment stored in EEPROM, and sends packet continuously.

Receiver firmware decodes packet, drives servo and ESC, controls lighting logic, and implements failsafe.

9. Control Logic & Automation Features

Headlights: toggled from transmitter.

Hazard: blinks all indicators and overrides turn signals.

Turn Indicators: triggered automatically based on steering direction.

Indicator Memory: continues blinking briefly after steering returns to center.

Brake Light: turns ON during sudden throttle decrease or stop.

Failsafe: stops motor, centers steering, enables hazard if packets are lost.

10. Testing & Results

The system was tested for steering/throttle response, headlight toggle, hazard blinking, indicator logic, brake light automation, EEPROM trim saving, and failsafe behavior. Results confirmed stable real-time performance.

11. Challenges Faced

Challenges included power stability issues, servo jitter, mechanical alignment, and calibration drift. These were managed using trim calibration, steering center learning, and stable grounding.

12. Conclusion

This project successfully demonstrates an ESP32-based smart RC vehicle implementing automotive-inspired lighting automation and safety features. ESP-NOW communication enabled smooth real-time control, while automation logic added realistic behavior similar to real vehicles.

13. Future Scope

- Wheel encoder feedback
- PID speed control
- Mobile dashboard
- GPS tracking
- Obstacle avoidance sensors

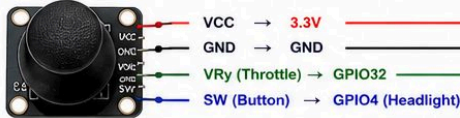
14. References

- Espressif ESP32 documentation
- ESP-NOW protocol reference
- Arduino IDE reference
- Servo and ESC PWM control resources

THROTTLE JOYSTICK (1-AXIS)

(Y-Axis for Throttle)

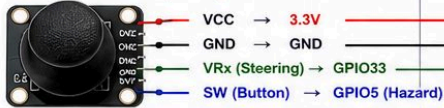
With Push Button (Headlight)



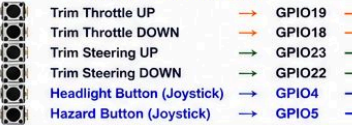
STEERING JOYSTICK (1-AXIS)

(X-Axis for Steering)

With Push Button (Hazard)

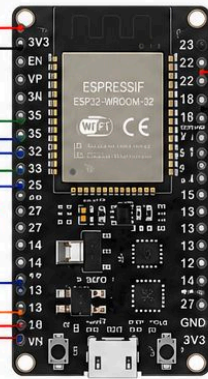


TRIM & FUNCTION BUTTONS



TRANSMITTER CIRCUIT DIAGRAM

(ESP32 REMOTE CONTROLLER)



POWER SUPPLY

3.7V Li-ion Battery

(>=2000mAh)



ESP-NOW
WIRELESS LINK

TRANSMITTER PIN CONNECTIONS

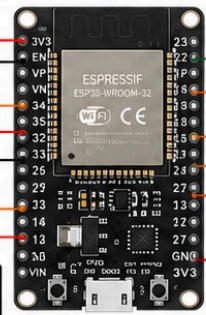
FUNCTION	GPIO (ESP32)
Throttle Analog (VRy)	GPIO32
Steering Analog (VRx)	GPIO33
Trim Throttle UP	GPIO19
Trim Throttle DOWN	GPIO18
Trim Steering UP	GPIO23
Trim Steering DOWN	GPIO22
Headlight Button (Joystick)	GPIO4
Hazard Button (Joystick)	GPIO5

TRANSMITTER NOTES:

- Joysticks are powered by 3.3V from ESP32.
- All buttons use internal pull-up (INPUT_PULLUP).
- All GNDs are common.
- Use at least 2000mAh battery for smooth functioning.

RECEIVER CIRCUIT DIAGRAM

(ESP32 RC CAR CONTROLLER)



POWER SUPPLY

3.7V Li-ion Battery

(>=2000mAh)



STEERING SERVO



ESC (SPEED CONTROLLER)



RECEIVER NOTES:

- Servo and ESC are powered directly from 3.7V battery.
- All LEDs are connected directly to GPIO pins (no resistors used).
- All grounds (GND) are common and connected together.



ALL GROUNDS CONNECTED TOGETHER

⚠️ USE AT LEAST 2000mAh BATTERY FOR SMOOTH FUNCTIONING

RECEIVER PIN CONNECTIONS

FUNCTION	GPIO (ESP32)
Steering Servo Signal	GPIO25
ESC Signal	GPIO26
Headlight (ON/OFF)	GPIO4
Front Left Indicator	GPIO13
Front Right Indicator	GPIO15
Rear Left Indicator	GPIO12
Rear Right Indicator	GPIO14
Brake Light	GPIO27

IMPORTANT NOTE:

READ LABELS ON RIGHT HAND SIDE FOR CORRECT WIRING.
DON'T GO WITH IMAGE.

